

Atividade 03 — Protótipo individual com ESP32 e Wokwi

Universidade Federal de Goiás — Instituto de Informática

Disciplina: Software para Sistemas Ubíquos — Prof. Dr. Otávio Calaça Xavier

1. Identificação do estudante e do projeto do grupo

Campo	Informação
Estudante	Matheus Vieira Mendes Pacheco
Matrícula	202302623
Projeto do grupo	<i>Não dê nem água</i> — cuidado assistido de plantas de interior
Integrantes do grupo	Matheus Vieira Mendes Pacheco, Davi Duarte Neco, Bárbara Nogueira
Recorte individual	Heartbeat e detecção de silêncio do sensor de ambiente (luz/temperatura)
Teste adversarial obrigatório	Final de matrícula 3 → <i>mudanças rápidas, ruído ou acionamentos repetidos</i>

Sobre o projeto do grupo

O sistema *Não dê nem água* monitora o microclima, a luminosidade e a umidade do solo de vasos domésticos, cruzando as leituras com os requisitos da espécie cadastrada para acionar irrigação e umidificação automáticas e emitir recomendações ao usuário. O sistema foi classificado, na Atividade 01, como IoT, Sistema Ciber-Físico e aplicação ubíqua, e seu **risco principal identificado foi a dependência de conectividade**: sem rede, o laço de controle não fecha e o usuário não é avisado.

Este protótipo individual implementa justamente o recorte que ataca esse risco: fazer com que a **interrupção do fluxo de dados se torne observável** no próprio dispositivo, em vez de passar despercebida.

2. Responsabilidade individual negociada

Perfil assumido: Comunicação e resiliência — contrato de evento, número de sequência, heartbeat, detecção de perda/silêncio, desconexão e recuperação.

O grupo negociou a divisão antes da implementação, de modo que cada integrante cobrisse uma parte distinta do sistema, com componentes, regras e atuadores diferentes:

Integrante	Perfil	Recorte implementado	Entrada principal	Atuador	Mecanismo de estado
Matheus V. M. Pacheco	Comunicação e resiliência	Heartbeat e silêncio do sensor de ambiente	LDR + DHT22	LED de status + buzzer	Validade do último dado + <i>debounce</i> de transição + <i>cooldown</i> do alerta
Davi Duarte Neco	Decisão e atuação	Irrigação por umidade do solo	Potenciômetro (substituto do higrômetro)	LED (bomba)	Histerese de dois limiares + <i>debounce</i> + <i>cooldown</i>
Bárbara Nogueira	Estado e processamento temporal	Expiração da validade do dado do reservatório	Potenciômetro + chave	Dois LEDs	Janela de amostras + expiração do dado

Como esta entrega se distingue das demais

- **Sensores diferentes:** é a única entrega que usa LDR e DHT22 (aquisição de duas grandezas ambientais distintas); os demais protótipos usam potenciômetro como entrada substituta.
- **Atuadores diferentes:** LED de status e buzzer, com alerta sonoro condicionado a *cooldown*.

- **Dois tipos de evento:** além do evento de dados (`LeituraAmbiente`), esta entrega emite um evento próprio de monitoramento (`EstadoConectividade`), que continua sendo publicado mesmo quando não há leituras — característica central do perfil de comunicação e resiliência.
- **Mecanismo de regra diferente:** enquanto o colega usa histerese sobre o valor medido, aqui a decisão é tomada sobre o **tempo decorrido desde o último evento**, com confirmação por ciclos consecutivos e limitação de taxa do atuador.

3. Relação do protótipo com as Atividades 01 e 02

Origem	Decisão anterior	Como aparece neste protótipo
Atividade 01 — item 3	Sensores de luminosidade (LDR/BH1750) e de temperatura/umidade do ar (DHT) no ponto do vaso	São exatamente as duas entradas físicas do circuito
Atividade 01 — item 5	Risco principal: dependência de conectividade; perda de conexão inviabiliza atuações e notificações	O protótipo detecta e sinaliza localmente a interrupção do fluxo de dados
Atividade 02 — itens 2 e 3	Contrato do evento <code>LeituraAmbiente</code> (produtor, entidade observada, tempo do evento, campos, unidade, sequence)	Reimplementado no formato JSON exigido, com número de sequência monotônico
Atividade 02 — item 4	Evento desatualizado é reconhecido comparando o tempo do evento com o instante atual	Implementado como <code>silencioAtual = agora - timestampUltimoEvento</code>
Atividade 02 — itens 10 e 11	Ao dispositivo (ESP32) couberam a validação local e o enfileiramento; à nuvem, a janela e a decisão global	Este recorte implementa apenas a parcela do dispositivo — sem Wi-Fi, MQTT ou nuvem
Atividade 02 — item 12	Falha escolhida: conexão indisponível , com degradação explícita e alerta na reconexão	Reproduzida pelo botão; o dispositivo sinaliza a perda e se recupera sozinho

4. Fenômeno, sensor/entrada, unidade e faixa

Fenômeno observado	Entrada	Pino	Unidade	Faixa válida
Luminosidade incidente no vaso	Módulo LDR (fotorresistor), saída analógica A0	GPI O 35	Leitura bruta do ADC	0 – 4095 (ADC de 12 bits, padrão do ESP32)
Temperatura do ar ao redor do vaso	DHT22	GPI O 15	°C	–40 a 80 °C (faixa do sensor); NaN indica leitura inválida
Interrupção do fluxo de dados (ocorrência)	Botão (entrada substituta)	GPI O 27	Booleano	Pressionado = HIGH (<code>INPUT_PULLDOWN</code>)
Tempo decorrido desde o último evento (grandeza derivada)	Calculada em software	—	ms	0 – ∞ (cresce enquanto não há leitura)

Declaração sobre entrada substituta

*O botão não é um sensor real de conectividade. Ele é uma **entrada substituta** que representa, de forma fictícia, a ocorrência "o sensor parou de reportar / o enlace de comunicação caiu". **Não houve aquisição nem validação do estado real de rede**, nem qualquer medição física de conectividade. A grandeza efetivamente medida pela regra é o tempo decorrido desde o último evento válido, essa sim calculada a partir do relógio interno do microcontrolador.*

Tratamento de leitura inválida ou ausente (requisito 5.1)

O protótipo trata **dois** casos distintos, o que reforça o perfil de resiliência:

1. **Leitura inválida:** se o DHT22 não devolver uma leitura válida, `dht.getTemperature()` retorna NaN. O programa verifica com `isnan()` e publica o campo como `"temperaturaC": null` — nunca um número inventado ou o último valor conhecido disfarçado de leitura atual.

2. **Leitura ausente:** quando nenhuma leitura chega, o evento `LeituraAmbiente` simplesmente deixa de ser emitido. O evento de monitoramento `EstadoConectividade`, porém, **continua sendo publicado a cada ciclo**, informando há quanto tempo o dispositivo está sem dados. Ou seja, a ausência de dado é ela mesma reportada, em vez de gerar silêncio total no barramento.

5. Circuito e componentes

Componentes

Id	Componente (tipo Wokwi)	Função no protótipo
esp	ESP32 DevKit-C V4 (board-esp32-devkit-c-v4)	Unidade de processamento
ldr1	Módulo fotorresistor (wokwi-photoresistor-sensor)	Luminosidade no ponto do vaso
dht1	DHT22 (wokwi-dht22)	Temperatura e umidade do ar
btn1	Botão (wokwi-pushbutton)	Entrada substituta: simula a perda de comunicação
led1	LED verde (wokwi-led) + r1 resistor 220 Ω	Status: aceso = dispositivo ativo
buzzer1	Buzzer (wokwi-buzzer)	Alerta sonoro na transição para silêncio

Ligações

De	Para	Observação
ldr1:VCC / ldr1:GND	esp:3V3 / esp:GND.1	Alimentação do módulo LDR
ldr1:AO	esp:35	Saída analógica → ADC
dht1:VCC / dht1:GND	esp:3V3 / esp:GND.2	Alimentação do DHT22
dht1:SDA	esp:15	Linha de dados do DHT22
btn1:1.1	esp:27	Pino de leitura, configurado como <code>INPUT_PULLDOWN</code>
btn1:2.1	esp:3V3	Pressionar o botão leva o pino a nível alto
led1:A → r1:1, r1:2	esp:26	LED de status com resistor limitador de 220 Ω
led1:C	esp:GND.3	Cátodo do LED
buzzer1:2 / buzzer1:1	esp:25 / esp:GND.3	Buzzer acionado por <code>tone()</code>
esp:TX / esp:RX	<code>\$serialMonitor:RX</code> / <code>\$serialMonitor:TX</code>	Monitor serial a 115200 bps

Biblioteca adicional

Biblioteca	Motivo
DHT sensor library for ESPx (header DHTesp.h)	Leitura do DHT22 no ESP32; as bibliotecas DHT genéricas não são confiáveis nessa plataforma

A biblioteca precisa ser instalada explicitamente pela aba **Library Manager** do Wokwi. Na primeira compilação sem ela, o build falha com fatal error: DHTesp.h: No such file or directory e o próprio simulador oferece o botão de instalação. O arquivo `libraries.txt` acompanha a entrega.

6. Contrato do evento

O protótipo emite **dois tipos de evento** no monitor serial, ambos em JSON e ambos respeitando o contrato mínimo exigido (`eventType`, `deviceId`, `entityId`, `eventTimeMs`, `sequence`, `value`, `unit`, `state` quando aplicável).

6.1 Evento `LeituraAmbiente` — evento de dados

Elemento	Valor
Produtor	esp32-vaso-matheus (gateway ESP32 do vaso)
Entidade observada	vaso-matheus-01 (microclima ao redor do vaso)
Tempo do evento	eventTimeMs — milissegundos desde o início da simulação
Sequência	sequence — contador monotônico, incrementado a cada leitura emitida
Campos e unidades	value (luminosidade, ADC bruto 0–4095), temperaturaC (°C, ou null se inválida)
Emissão	A cada ciclo de 150 ms, somente quando o dispositivo está "falando"

```
{ "eventType": "LeituraAmbiente", "deviceId": "esp32-vaso-matheus", "entityId": "vaso-matheus-01",
  "eventTimeMs": 1503, "sequence": 10, "value": 2048, "unit": "raw_adc", "temperaturaC": 29.8 }
```

6.2 Evento EstadoConectividade — evento de monitoramento (heartbeat)

Elemento	Valor
Produtor	esp32-vaso-matheus (o mesmo dispositivo, em papel de monitor)
Entidade observada	vaso-matheus-01
Tempo do evento	eventTimeMs — milissegundos desde o início da simulação
Sequência	sequence — contador próprio , independente do contador de leituras
Campos e unidades	value = tempo decorrido desde o último evento de leitura (unit: ms_desde_ultimo_evento)
Estado	state assume ATIVO ou SILENCIOSO
Emissão	A cada ciclo de 150 ms, sempre — inclusive quando não há leituras

```
{ "eventType": "EstadoConectividade", "deviceId": "esp32-vaso-matheus", "entityId": "vaso-matheus-01",
  "eventTimeMs": 1505, "sequence": 10, "value": 2, "unit": "ms_desde_ultimo_evento", "state": "ATIVO" }
```

Por que dois contadores de sequência

Cada fluxo tem seu próprio `sequence` porque são fluxos independentes: o de leituras **para** quando o sensor cala, enquanto o de monitoramento **continua**. A diferença entre os dois contadores ao longo do tempo é, por si só, uma medida de quantos ciclos o dispositivo passou sem dados — informação que seria perdida se ambos compartilhassem um único contador.

Limitação declarada: o protótipo não está sincronizado com relógio externo (sem NTP). Portanto `eventTimeMs` representa o tempo decorrido **desde o início da simulação** (`millis()`), e não um instante absoluto. Em produção, conforme modelado na Atividade 02, o carimbo de tempo seria gerado com o relógio sincronizado por NTP, permitindo janelas por tempo de evento no servidor.

7. Estado, regra e atuação

7.1 Estado mantido entre ciclos

Variável	Tipo	Papel
timestampUltimoEvento	unsigned long	Instante da última leitura emitida — base de todo o cálculo de validade
estadoSilencioso	bool	Estado corrente da máquina de estados (ATIVO / SILENCIOSO)
ciclosConsecutivosSilencioso	int	Ciclos seguidos em que a condição de silêncio se manteve (<i>debounce</i>)
ciclosConsecutivosAtivo	int	Ciclos seguidos em que a condição de atividade se manteve (<i>debounce</i>)

Variável	Tipo	Papel
timestampUltimoAlerta	unsigned long	Instante do último bipe — base do <i>cooldown</i> do atuador
seqLeitura / seqHeartbeat	uint32_t	Números de sequência dos dois fluxos de evento

7.2 Parâmetros da regra

Parâmetro	Valor	Significado
Ciclo do laço	150 ms	Período de amostragem e de verificação
LIMIAR_SILENCIO_MS	600 ms	Acima disso sem nova leitura, a condição "parece silencioso" fica verdadeira
CICLOS_DEBOUNCE	4 ciclos (≈ 600 ms)	Ciclos consecutivos necessários para confirmar qualquer transição de estado
COOLDOWN_ALERTA_MS	2000 ms	Intervalo mínimo entre dois bipes do buzzer

Tempos resultantes: a transição ATIVO → SILENCIOSO leva ≈ 600 ms (limiar) + 4 ciclos de confirmação ≈ **1,2 s** de interrupção contínua. A volta SILENCIOSO → ATIVO leva ≈ **0,6 s** após a primeira leitura voltar a chegar. **Consequência direta: qualquer interrupção mais curta que ≈ 1,2 s não altera o estado nem aciona o buzzer.**

Esses tempos foram confirmados experimentalmente: na execução registrada na seção 8.2, a transição ocorreu com `value = 1139 ms`, e o ciclo real medido foi de 161 ms — ligeiramente acima dos 150 ms do `delay()`, por causa do tempo de leitura dos sensores e de impressão dos eventos na serial.

7.3 A regra (não é comparação instantânea)

A decisão combina **três** características de estado/qualidade, nenhuma delas um simples `valor > limite`:

1. **Validade/expiração do último dado** — a condição não é avaliada sobre o valor medido, e sim sobre o *tempo decorrido* desde o último evento válido.
2. **Debounce da transição** — a condição precisa se manter por 4 ciclos consecutivos para que o estado mude, em qualquer das duas direções.
3. **Cooldown do atuador** — mesmo com uma transição legítima, o alerta sonoro só dispara se já houver passado o intervalo mínimo desde o bipe anterior.

```

ESTADO: timestampUltimoEvento, estadoSilencioso,
        ciclosConsecutivosSilencio, ciclosConsecutivosAtivo, timestampUltimoAlerta

A CADA CICLO (150 ms):

    SE o dispositivo esta "falando":                // botao nao pressionado
        ler LDR e DHT22
        SE temperatura e NaN: publicar temperaturaC = null // leitura invalida
        seqLeitura++
        timestampUltimoEvento = agora
        EMITIR evento LeituraAmbiente

    // ---- monitoramento: roda SEMPRE, mesmo sem leitura ----
    silencioAtual = agora - timestampUltimoEvento // validade do ultimo dado
    pareceSilencioso = silencioAtual > LIMIAIR_SILENCIO_MS // 600 ms

    SE pareceSilencioso: ciclosConsecutivosSilencio++; ciclosConsecutivosAtivo = 0
    SENA0:                  ciclosConsecutivosAtivo++;   ciclosConsecutivosSilencio = 0

    SE (NAO estadoSilencioso) E (ciclosConsecutivosSilencio >= 4): // debounce
        estadoSilencioso = VERDADEIRO
        LED de status <- APAGADO
        SE (agora - timestampUltimoAlerta >= COOLDOWN_ALERTA_MS): // cooldown
            bipar buzzer (1 kHz, 200 ms)
            timestampUltimoAlerta = agora

    SENA0 SE (estadoSilencioso) E (ciclosConsecutivosAtivo >= 4): // debounce
        estadoSilencioso = FALSO
        LED de status <- ACESO

    seqHeartbeat++
    EMITIR evento EstadoConectividade { value: silencioAtual, state: ATIVO|SILENCIOSO }

```

7.4 Máquina de estados

Estado	Condição de entrada	LED	Buzzer	Significado
ATIVO	Estado inicial, ou 4 ciclos seguidos com dado fresco	Aceso	—	O dispositivo está recebendo dados do sensor
SILENCIOSO	4 ciclos seguidos com silencioAtual > 600 ms	Apagado	Bipe na transição, respeitando o <i>cooldown</i>	O fluxo de dados parou; o dado atual não é confiável

7.5 Atuação e comportamento seguro

Situação	Resposta observável
Fluxo de dados normal	LED de status aceso; eventos state: "ATIVO"
Interrupção confirmada (> ≈1,2 s)	LED apaga + um bipe; eventos state: "SILENCIOSO"
Interrupções curtas / ruído	Nenhuma mudança de LED e nenhum bipe
Transições repetidas em menos de 2 s	O estado muda e é registrado, mas o bipe é suprimido pelo <i>cooldown</i>
Leitura do DHT22 inválida	Publica "temperaturaC": null — não inventa valor nem repete o anterior
Sem nenhuma leitura chegando	O heartbeat continua publicando e informa a idade do último dado

O princípio de segurança adotado é o de **não afirmar mais do que se sabe**: na ausência de dado válido, o sistema não mantém a última informação como se ainda fosse atual — ele declara explicitamente que está sem dados.

8. Resultados dos três testes

Os três testes foram executados no Wokwi, no projeto wokwi.com/projects/475464708123834369. Os trechos de monitor serial reproduzidos abaixo foram copiados literalmente das execuções registradas nas imagens da pasta [evidencias/](#).

Uma observação de medição, válida para os três testes: o período real do ciclo é de ≈ 161 ms, e não exatamente os 150 ms do `delay()`. A diferença corresponde ao tempo de leitura dos sensores e de impressão dos dois eventos JSON na serial. Todos os tempos discutidos a seguir usam o valor medido.

8.1 Teste 1 — Estado normal

Item	Descrição
Estado inicial	Simulação recém-iniciada; botão solto; LED de status aceso; <code>estadoSilencioso = falso</code>
Sequência de entradas	Nenhuma intervenção — simulação correndo livremente por ≈ 10 s
Valores e tempos relevantes	Ciclo medido de 161 ms (<code>eventTimeMs</code> 10073 $\rightarrow$ 10246); <code>value</code> do heartbeat constante em 12 ms ; LDR estável em 1001 (ADC bruto); DHT22 em 29,8 °C; os dois contadores de sequência avançam em paralelo (56, 57)
Resultado esperado	LED permanece aceso, buzzer em silêncio, todos os eventos com <code>"state": "ATIVO"</code> ; <code>LeituraAmbiente</code> e <code>EstadoConectividade</code> alternando no monitor serial
Resultado observado	Exatamente o esperado (trecho abaixo): os dois tipos de evento alternam a cada ciclo, o heartbeat reporta apenas 12 ms desde a última leitura e o estado permanece ATIVO durante toda a execução. O LED de status ficou aceso e o buzzer não foi acionado
Explicação da decisão	Como há leitura nova a cada ciclo, <code>silencioAtual</code> (12 ms) nunca se aproxima do limiar de 600 ms; <code>ciclosConsecutivosSilencio</code> permanece em zero e a regra nunca é satisfeita. O sistema corretamente não atua
Evidência	evidencias/teste-normal.png

```
{ "eventType": "LeituraAmbiente", "deviceId": "esp32-vaso-matheus", "entityId": "vaso-matheus-01", "eventTimeMs": 10073, "sequence": 56, "value": 1001, "unit": "raw_adc", "temperaturaC": 29.8 }
{ "eventType": "EstadoConectividade", "deviceId": "esp32-vaso-matheus", "entityId": "vaso-matheus-01", "eventTimeMs": 10084, "sequence": 56, "value": 12, "unit": "ms_desde_ultimo_evento", "state": "ATIVO" }
{ "eventType": "LeituraAmbiente", "deviceId": "esp32-vaso-matheus", "entityId": "vaso-matheus-01", "eventTimeMs": 10246, "sequence": 57, "value": 1001, "unit": "raw_adc", "temperaturaC": 29.8 }
{ "eventType": "EstadoConectividade", "deviceId": "esp32-vaso-matheus", "entityId": "vaso-matheus-01", "eventTimeMs": 10257, "sequence": 57, "value": 12, "unit": "ms_desde_ultimo_evento", "state": "ATIVO" }
```

8.2 Teste 2 — Decisão e atuação

Item	Descrição
Estado inicial	Simulação em regime normal, <code>state = ATIVO</code> , LED aceso
Sequência de entradas	Clicar e manter pressionado o botão por ≈ 2 s; observar a transição; soltar e observar a recuperação
Valores e tempos relevantes	<code>value</code> cresce de forma contínua a cada ciclo: 817 $\rightarrow$ 978 $\rightarrow$ 1139 $\rightarrow$ 1302 ms. A transição para SILENCIOSO ocorreu no evento de sequência 69, com <code>value = 1139</code> ms — valor previsto pelo projeto: 600 ms de limiar + 4 ciclos de 161 ms ≈ 1244 ms
Resultado esperado	Após $\approx 1,2$ s o LED apaga, o buzzer bipa uma vez e os eventos passam a <code>"state": "SILENCIOSO"</code> ; ao soltar o botão, <code>LeituraAmbiente</code> volta a ser emitido e o estado retorna a "ATIVO"
Resultado observado	A transição foi capturada no instante exato (trecho abaixo). Dois pontos confirmam a regra: (a) os eventos de sequência 67 e 68 já estão com <code>value</code> acima do limiar (817 e 978 ms) e ainda assim permanecem ATIVO , porque o <i>debounce</i> de 4 ciclos não havia se completado; (b) no ciclo seguinte, com <code>value = 1139</code> ms, o quarto ciclo consecutivo se cumpre e o estado muda para SILENCIOSO, com o LED apagando e um bipa único. Note também que nenhum evento <code>LeituraAmbiente</code> aparece nesse intervalo — o sensor está calado, e apenas o heartbeat continua publicando
Explicação da decisão	Sem leituras, <code>silencioAtual</code> ultrapassa 600 ms; a condição se mantém por 4 ciclos consecutivos e só então a transição é confirmada, acionando LED e buzzer. A diferença entre o valor previsto (≈ 1244 ms) e o medido (1139 ms) decorre de o limiar ter sido cruzado no meio de um ciclo, e não exatamente no seu início
Evidência	evidencias/teste-decisao.png

```
{
  "eventType": "EstadoConectividade",
  "deviceId": "esp32-vaso-matheus",
  "entityId": "vaso-matheus-01",
  "eventTimeMs": 11933,
  "sequence": 67,
  "value": 817,
  "unit": "ms_desde_ultimo_evento",
  "state": "ATIVO"
}
{
  "eventType": "EstadoConectividade",
  "deviceId": "esp32-vaso-matheus",
  "entityId": "vaso-matheus-01",
  "eventTimeMs": 12094,
  "sequence": 68,
  "value": 978,
  "unit": "ms_desde_ultimo_evento",
  "state": "ATIVO"
}
{
  "eventType": "EstadoConectividade",
  "deviceId": "esp32-vaso-matheus",
  "entityId": "vaso-matheus-01",
  "eventTimeMs": 12255,
  "sequence": 69,
  "value": 1139,
  "unit": "ms_desde_ultimo_evento",
  "state": "SILENCIOSO"
}
{
  "eventType": "EstadoConectividade",
  "deviceId": "esp32-vaso-matheus",
  "entityId": "vaso-matheus-01",
  "eventTimeMs": 12418,
  "sequence": 70,
  "value": 1302,
  "unit": "ms_desde_ultimo_evento",
  "state": "SILENCIOSO"
}
```

8.3 Teste 3 — Adversarial (final de matrícula 3: mudanças rápidas, ruído ou acionamentos repetidos)

Item	Descrição
Estado inicial	Simulação em regime normal, state = ATIVO, LED aceso, buzzer em silêncio
Sequência de entradas	Série de acionamentos curtos e sucessivos do botão (aperta e solta, repetidamente), cada um bem mais breve que o tempo necessário para confirmar uma transição ($\approx 1,2$ s), simulando um enlace instável
Valores e tempos relevantes	Ciclo medido de 161 ms, constante (eventTimeMs 32357 $\rightarrow$ 32518 $\rightarrow$ 32679 $\rightarrow$ 32840). Durante a interrupção capturada, value cresce 173 $\rightarrow$ 334 $\rightarrow$ 495 $\rightarrow$ 656 ms . O último valor está acima do limiar de 600 ms , mas corresponde a apenas o primeiro ciclo nessa condição — três a menos do que o CICLOS_DEBOUNCE exige
Resultado esperado	O LED permanece aceso , o buzzer não bipa e os eventos continuam com "state": "ATIVO", mesmo quando o tempo sem leitura ultrapassa momentaneamente o limiar de validade
Resultado observado	Exatamente o esperado (trecho abaixo). O ponto decisivo é o evento de sequência 192: value = 656 ms, portanto o limiar de 600 ms já foi cruzado , e o estado permanece "ATIVO". Ou seja, cruzar o limiar de validade não é suficiente para mudar o estado — é preciso que a condição se sustente pelos quatro ciclos de confirmação, o que não ocorreu. Nenhum evento LeituraAmbiente aparece na janela (o sensor está momentaneamente calado) e nenhum evento reporta SILENCIOSO. O LED permaneceu aceso e o buzzer não foi acionado
Explicação da decisão	A cada ciclo em que silencioAtual > 600 ms, o contador ciclosConsecutivosSilencio é incrementado; qualquer leitura que volte a chegar o zera. Como as interrupções são curtas, o contador nunca alcança 4 e a regra jamais é satisfeita. O ruído é registrado no evento (o campo value sobe e desce, preservando o histórico), mas não se converte em decisão nem em atuação
Evidência	evidencias/teste-adversarial.png

```
{
  "eventType": "EstadoConectividade",
  "deviceId": "esp32-vaso-matheus",
  "entityId": "vaso-matheus-01",
  "eventTimeMs": 32357,
  "sequence": 189,
  "value": 173,
  "unit": "ms_desde_ultimo_evento",
  "state": "ATIVO"
}
{
  "eventType": "EstadoConectividade",
  "deviceId": "esp32-vaso-matheus",
  "entityId": "vaso-matheus-01",
  "eventTimeMs": 32518,
  "sequence": 190,
  "value": 334,
  "unit": "ms_desde_ultimo_evento",
  "state": "ATIVO"
}
{
  "eventType": "EstadoConectividade",
  "deviceId": "esp32-vaso-matheus",
  "entityId": "vaso-matheus-01",
  "eventTimeMs": 32679,
  "sequence": 191,
  "value": 495,
  "unit": "ms_desde_ultimo_evento",
  "state": "ATIVO"
}
{
  "eventType": "EstadoConectividade",
  "deviceId": "esp32-vaso-matheus",
  "entityId": "vaso-matheus-01",
  "eventTimeMs": 32840,
  "sequence": 192,
  "value": 656,
  "unit": "ms_desde_ultimo_evento",
  "state": "ATIVO"
}
```

Análise exigida para o teste adversarial

1. Por que uma implementação ingênua tomaria uma decisão incorreta. A forma mais direta de escrever essa regra seria se (nao chegou leitura neste ciclo) entao alertar. Com essa lógica, **cada toque no botão** — e, em campo, cada micro-oscilação do enlace, cada pacote perdido, cada atraso momentâneo do sensor — produziria imediatamente uma transição de estado e um acionamento do alerta. O resultado seria um LED piscando sem parar e um buzzer disparando dezenas de vezes em poucos segundos, para um problema que **não existe**: o sensor continua entregando dados. Além de inútil, o alerta perderia credibilidade — o usuário passaria a ignorá-lo justamente quando uma falha real acontecesse. Em um sistema ubíquo, cuja atuação deve ser discreta, esse comportamento é uma falha de projeto tão grave quanto deixar de detectar a falha verdadeira.

2. Qual mecanismo foi implementado para tratar o problema. Foram implementadas duas proteções em camadas distintas:

- **Debounce da transição de estado** (`CICLOS_DEBOUNCE = 4`): a condição de silêncio precisa se manter verdadeira por 4 ciclos consecutivos (≈ 600 ms) para que o estado mude. Qualquer leitura que chegue no meio do caminho zera o contador. Somado ao limiar de validade de 600 ms, isso significa que apenas interrupções contínuas acima de $\approx 1,2$ s são reconhecidas como silêncio real. Esse mecanismo atua sobre a **decisão**.
- **Cooldown do alerta** (`COOLDOWN_ALERTA_MS = 2000`): mesmo quando a transição é legítima, o bipe só é emitido se houver decorrido ao menos 2 s desde o anterior. Se várias transições legítimas ocorrerem em sequência rápida, o estado é atualizado e registrado normalmente, mas o alerta sonoro não se acumula. Esse mecanismo atua sobre a **atuação**.

A separação é proposital: o *debounce* impede que ruído vire decisão; o *cooldown* impede que decisões legítimas, porém frequentes, virem incômodo. Ambos preservam o registro completo no monitor serial — nada é escondido, apenas a atuação física é contida.

3. Qual foi a saída observada durante o teste. Durante a rajada de acionamentos, o campo `value` dos eventos de heartbeat subiu e desceu continuamente, acompanhando cada interrupção, enquanto o campo `state` permaneceu em "ATIVO" em todos os eventos da janela capturada. O caso mais significativo está na sequência 192, com `value = 656 ms`: o tempo sem leitura já havia **ultrapassado o limiar de validade de 600 ms** e, mesmo assim, o estado não mudou — porque aquele era apenas o primeiro ciclo na condição de silêncio, e a regra exige quatro consecutivos. O LED de status permaneceu aceso e o buzzer não foi acionado nenhuma vez durante todo o teste. É exatamente o ponto em que uma implementação ingênua já teria apagado o LED e disparado o alerta, tratando uma oscilação momentânea como falha do sensor.

*Observação sobre a cobertura deste teste. A janela registrada em `evidencias/teste-adversarial.png` documenta uma das interrupções da rajada, incluindo o cruzamento do limiar de validade sem mudança de estado — que é o comportamento crítico a ser demonstrado. Uma captura complementar, mostrando o campo `value` retornando a ≈ 12 ms entre duas excursões sucessivas, documentaria adicionalmente o caráter **repetido** dos acionamentos.*

9. Limitações da simulação

#	Limitação	O que exigiria hardware ou ambiente reais
1	O botão é uma entrada substituta : não há medição real de conectividade	Um enlace Wi-Fi/MQTT de verdade, com perdas, atrasos e reconexões reais
2	<code>eventTimeMs</code> é <code>millis()</code> — tempo desde o início da simulação, sem relógio sincronizado	Sincronização por NTP para carimbar eventos em tempo absoluto e permitir janelas por tempo de evento na nuvem
3	Os tempos foram propositalmente reduzidos (limiar de 600 ms, ciclo de 150 ms, <i>cooldown</i> de 2 s) para viabilizar o teste manual	Em produção, o limiar de silêncio de um gateway doméstico seria de minutos; a lógica da regra é idêntica, apenas as constantes mudam
4	A luminosidade é reportada como leitura bruta do ADC , sem conversão calibrada para lux	Calibração do divisor resistivo do LDR ou uso de um sensor digital (BH1750)
5	O DHT22 simulado não reproduz falhas reais de CRC, <i>timing</i> ou fiação; o caminho de leitura inválida (NaN) foi implementado, mas é difícil de exercitar no simulador	Hardware real, onde falhas de leitura do DHT22 ocorrem naturalmente
6	LED e buzzer representam a notificação ao usuário	No sistema completo, o alerta seria push no aplicativo e aviso por voz pela Alexa, conforme a Atividade 01
7	Não há Wi-Fi, MQTT nem nuvem neste recorte	Integração prevista como evolução, conforme a distribuição definida na Atividade 02
8	O estouro de <code>millis()</code> (≈ 49 dias) não é tratado	Relevante apenas em operação contínua de longa duração

10. Declaração de uso de inteligência artificial generativa

Ferramenta utilizada: Claude (Anthropic), via Claude Code.

Partes da atividade em que foi utilizada:

- Discussão e negociação da divisão de responsabilidades entre os integrantes do grupo;
- Estruturação inicial do código do protótipo (`sketch.ino`) e do circuito (`diagram.json`);
- Levantamento dos nomes corretos de componentes e pinos na documentação do Wokwi;
- Redação e organização deste relatório.

Síntese das orientações fornecidas: Foram fornecidos os enunciados das Atividades 01, 02 e 03 e solicitado (i) um recorte coerente com o perfil de comunicação e resiliência dentro do projeto do grupo; (ii) um protótipo com evento JSON, estado e atuação que não dependesse de comparação instantânea; (iii) adequação do protótipo ao teste adversarial correspondente ao final de matrícula 3; e (iv) a estruturação do relatório conforme o item 9.1 do enunciado.

Sugestões aceitas:

- A escolha do recorte (heartbeat e detecção de silêncio) e o uso de LDR + DHT22, por manterem continuidade com o risco principal identificado na Atividade 01;
- A separação em dois tipos de evento, com contadores de sequência independentes;
- A combinação de *debounce* na decisão e *cooldown* na atuação como resposta ao teste adversarial.

Sugestões alteradas:

- Os parâmetros temporais originalmente propostos (limiar de silêncio de 5 s e ciclo de 500 ms) foram reduzidos para 600 ms e 150 ms, porque na prática **inviabilizavam o teste adversarial**: com aqueles valores, seria necessário manter o botão pressionado por mais de 6 s para observar qualquer transição, e a oscilação rápida exigida pelo teste não produzia efeito observável algum;
- O trecho de circuito com relé, sugerido inicialmente como atuador, foi descartado em favor de LED + buzzer, por serem diretamente observáveis na simulação.

Sugestões rejeitadas:

- A proposta inicial de reaproveitar potenciômetro/umidade do solo como entrada foi rejeitada por gerar sobreposição com a entrega de outro integrante do grupo, o que o enunciado veda.

Erros, limitações ou decisões inadequadas encontrados na resposta da ferramenta:

1. **Erro de parametrização (o mais relevante).** A ferramenta dimensionou os tempos da regra pensando em um sistema real (limiar de 5 s), sem considerar que o teste seria executado manualmente com o mouse. O resultado foi um protótipo *correto em lógica, porém não testável na prática*: cliques rápidos não produziam nenhum efeito observável. O problema só apareceu ao tentar executar o teste, e exigiu redimensionar os parâmetros — evidência de que a validação experimental não pode ser substituída pela revisão do código.
2. **Afirmção incorreta sobre dependências.** A ferramenta indicou que a biblioteca do DHT22 seria resolvida automaticamente pelo Wokwi. Na prática, a compilação falhou com `fatal error: DHTesp.h: No such file or directory`, sendo necessário instalá-la explicitamente pela aba Library Manager.
3. **Limitação de verificação.** A ferramenta não conseguiu executar a simulação até o fim para confirmar o funcionamento: todas as suas tentativas de compilação, em sessão anônima, foram interrompidas pela mensagem *"Build Servers Busy"* do Wokwi. O projeto compilou normalmente quando executado por mim, já autenticado na minha conta. Toda a verificação experimental do comportamento — os três testes e as capturas de tela — foi feita por mim.
4. **Hipótese diagnóstica equivocada.** Diante das falhas de compilação, a ferramenta levantou a hipótese de que a causa seria a biblioteca do DHT22, apoiada no fato de que um protótipo sem bibliotecas externas havia compilado em poucos segundos. Ao testar essa hipótese removendo a biblioteca, a compilação continuou lenta, refutando a explicação: o problema era apenas a fila compartilhada do simulador. O episódio mostra que hipóteses plausíveis da ferramenta precisam ser testadas antes de aceitas.

Declaro que compreendo integralmente o código entregue, que executei e testei o protótipo, e que sou capaz de explicar cada variável de estado, cada parâmetro da regra e o comportamento observado em cada um dos três testes.

11. Link do projeto no Wokwi

Os arquivos enviados no ZIP constituem a versão oficial da entrega; o link serve para facilitar a execução e a inspeção.

12. Checklist de entrega

- [x] O protótipo está relacionado ao projeto do grupo
- [x] A responsabilidade individual está declarada (seção 2)
- [x] A implementação é distinguível das entregas dos demais integrantes (seção 2)
- [x] O circuito executa no Wokwi
- [x] Existe pelo menos uma entrada e uma saída observável
- [x] O monitor serial apresenta evento JSON com sequência (seção 6)
- [x] A regra utiliza estado, tempo e qualidade — não apenas comparação instantânea (seção 7)
- [x] Os três testes foram executados e documentados (seção 8)
- [x] O teste adversarial corresponde ao final da matrícula (3 → ruído/acionamentos repetidos)
- [x] As limitações da simulação estão explícitas (seção 9)
- [x] O uso de IA está declarado (seção 10)
- [x] O ZIP contém `sketch.ino`, `diagram.json`, `libraries.txt`, `relatorio.pdf` e `evidencias/`
- [x] O link do Wokwi está acessível (seção 11)